COMSATS University Islamabad, Vehari Campus, Pakistan

Department of Computer Science

Ghost Interview (AI Mock Interviewer)



Software Design Document (SDD)

Bachelor of Science in Computer Science

Submitted By

Ehsan Hamza SP23-BCS-019

Shaharyar Altaf SP23-BCS-055

Supervisor

Sir Qizar Javed

Contents

4.1 Introduction to Software Design 3
4.2 Design Methodology and Software Process Model 3
4.2.1 Design Methodology 3
4.2.2 Software Process Model 4
4.3 System Overview 4
4.4 Architectural Design 5
4.5 Design Models 6
4.5.1 Activity Diagrams 6
4.5.2 Class Diagram 9
4.5.3 Sequence Diagrams 12
4.5.4 State Transition Diagrams 14
4.6 Data Flow Diagrams 14
4.6.1 Level 1 Data Flow Diagram 15
4.6.2 Level 2 Data Flow Diagram 15
4.7 Data Design 16
4.7.1 Entity Relationship Diagram (ERD) 17
4.7.2 Database Architecture 18
4.8 Database Schema Diagram 19
4.9 Database Tables / Collections 19
4.9.1 User Collection 21
4.9.2 Core Data Collections 22
4.9.3 Relationships Between Collections 22
4.9.4 Security and Data Integrity 23
4.9.5 Scalability Considerations 23
4.10 Chapter Summary 23

Chapter 4

Software Design

4.1 Introduction to Software Design

This chapter presents the detailed software design of the Ghost Interview AI Mock Interviewer system. It translates the requirements identi ed in Chapter 3 into structured architectural, behavioral, and data models. The purpose of this chapter is to provide a clear blueprint for system implementation, ensuring maintainability, scalability, and modular development.

This chapter includes architectural design, UML models, data ow models, and database design for the Ghost Interview web-based AI application. The system integrates multiple AI services including OpenAI Whisper, Gemini API, DeepFace/Hume AI, and ElevenLabs to deliver a comprehensive mock interview experience.

4.2 Design Methodology and Software Process Model

4.2.1 Design Methodology

This project follows the Object-Oriented Design (OOD) methodology combined with a Layered Architecture approach. This methodology is selected due to the following reasons:

Modularity: Each system module (User Management, AI Interview, Speech Analysis, Emotion Detection, Reporting) is developed as an independent, reusable component.

Scalability: The layered architecture allows each tier (frontend, backend, AI services) to be scaled independently based on load.

Loose Coupling: AI service integrations (Whisper, Gemini, ElevenLabs, DeepFace) are loosely coupled through API abstraction layers, allowing easy replacement or upgrade of individual services.

Reusability: Common components such as authentication, session management, and report generation are reused across modules.

The design follows key principles including SOLID, DRY (Don’t Repeat Yourself), and Separation of Concerns, ensuring clean and maintainable code architecture.

4.2.2 Software Process Model

The project follows the Agile Incremental development model. This model is selected for the following reasons:

The system integrates multiple AI APIs whose behavior may require iterative re nement based on real testing results.

Features such as emotion detection accuracy and speech analysis quality need continuous evaluation and improvement cycles.

Agile allows the team to deliver working modules incrementally: rst user management, then the interview engine, then analysis modules, and nally the reporting dashboard.

Testing and validation are integrated at the end of each sprint, reducing late-stage defect discovery.

Development Phases / Sprints:

1. Sprint 1: User registration, login, and dashboard setup.
2. Sprint 2: Job role selection and Gemini API interview question integration.
3. Sprint 3: AI avatar voice interaction using ElevenLabs and WebRTC.
4. Sprint 4: Speech-to-text conversion (Whisper) and ller word detection.
5. Sprint 5: Emotion detection (DeepFace/Hume AI) and sentiment analysis.
6. Sprint 6: Con dence & Competency Report generation and interview history dashboard.
7. Sprint 7: Testing, optimization, and nal deployment.

4.3 System Overview

This section provides a high-level overview of Ghost Interview system components and their interactions with external entities.

External Actors:

Candidate (End User): Interacts with the system through a web browser using webcam and microphone.

Administrator: Manages system con guration and monitors users.

Gemini API: Provides AI-generated interview questions based on job role.

OpenAI Whisper API: Converts candidate’s speech to text.

DeepFace / Hume AI: Analyzes facial emotions from webcam frames.

ElevenLabs API: Synthesizes realistic AI avatar voice for questions.

Database: Stores user data, session transcripts, and generated reports.

Major Subsystems:

User Management Subsystem

AI Interview Engine Subsystem

Speech & Sentiment Analysis Subsystem

Emotion Detection Subsystem

Reporting & Analytics Subsystem

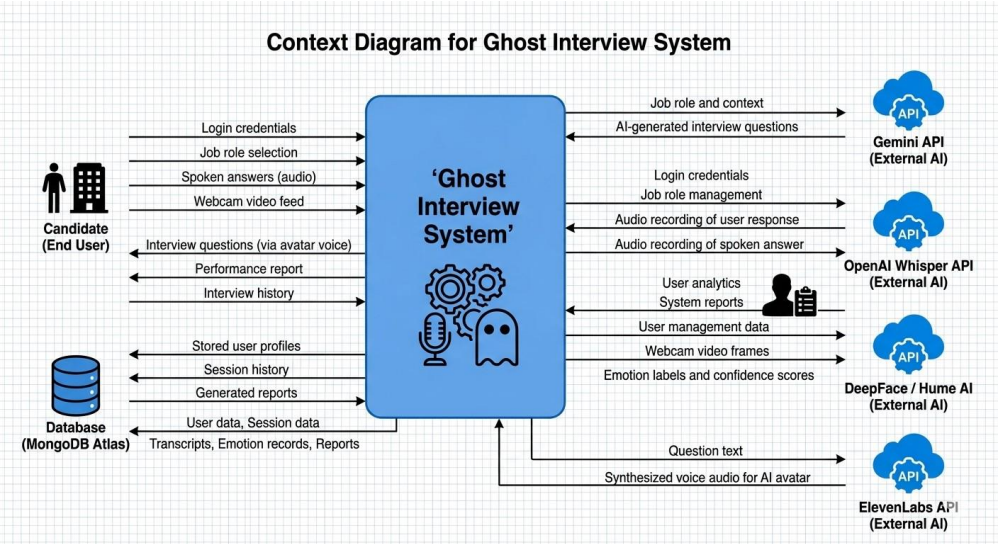


Figure 4.1: Context Diagram of Ghost Interview System

4.4 Architectural Design

This section describes the high-level three-tier architecture of the Ghost Interview system. The system is divided into the following layers:

Presentation Layer (Frontend): Built using React.js (v18+). Handles all user interface components including login, job role selection, interview screen (webcam + avatar), and report dashboard. WebRTC is used for real-time webcam and microphone access.

Business Logic Layer (Backend): Built using Node.js (v20+). Handles session management, API orchestration, speech analysis pipeline, emotion data aggregation, and report computation. RESTful APIs are exposed to the frontend.

AI Services Layer: Comprises four external AI APIs:

Gemini API Interview question generation.

OpenAI Whisper Speech-to-text conversion.

DeepFace / Hume AI Facial emotion detection.

ElevenLabs AI voice synthesis for avatar.

Data Layer (Database): Stores all persistent data including user accounts, interview sessions, transcripts, emotion metrics, and generated reports. A NoSQL database (e.g., MongoDB) is preferred for its exibility with session and report data structures.

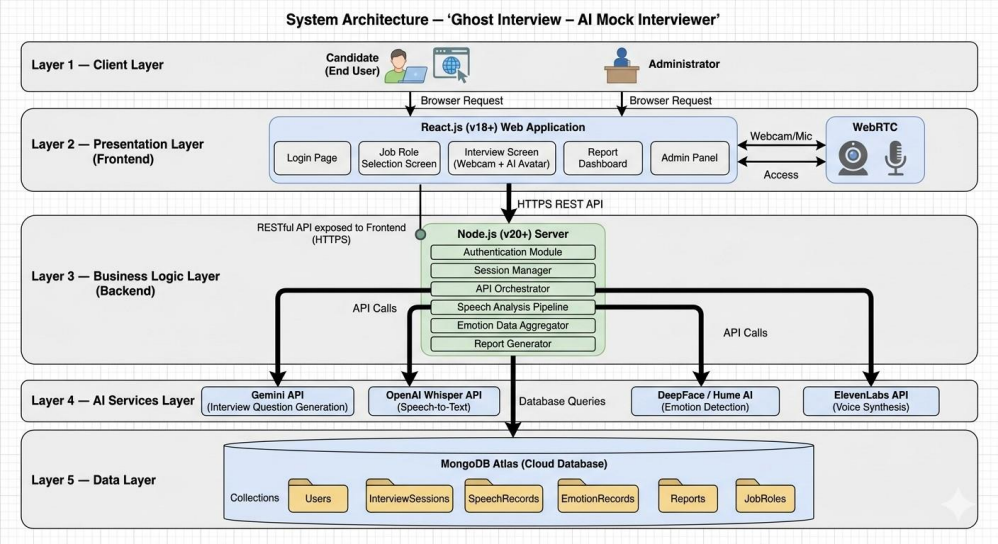


Figure 4.2: System Architecture Diagram of Ghost Interview

Component Interaction: The frontend communicates with the backend via RESTful APIs over HTTPS. The backend orchestrates calls to the four AI service APIs, processes responses, and stores results in the database. All API keys are stored server-side for security. WebRTC streams webcam and audio data directly in the browser, which is then captured and forwarded to the backend for processing.

4.5 Design Models

This section presents detailed UML design models of the Ghost Interview system. These models describe system behavior, structure, and interactions, helping developers understand system work ow before implementation.

4.5.1 Activity Diagrams

Activity diagrams represent the work ow of the di erent modules of the system.

Module 1: User Management

Description: This diagram illustrates the user registration, login, authentication, and pro le management processes for Ghost Interview.

Flow:

1. User navigates to the registration or login page.
2. For registration: User enters name, email, and password → System validates input → Account created → Con rmation displayed.
3. For login: User enters credentials → System validates against database → If valid, session created and user redirected to dashboard → If invalid, error message displayed.
4. User can update pro le information from the dashboard.

Module 2: AI Interview Core Module

Description: This diagram illustrates the core interview session work ow including job role selection, avatar interaction, speech capture, and analysis.

Flow:

1. User selects a job role from prede ned categories.
2. System requests interview questions from Gemini API.
3. Webcam and microphone permissions are requested via WebRTC.
4. AI avatar delivers the rst question using ElevenLabs voice.

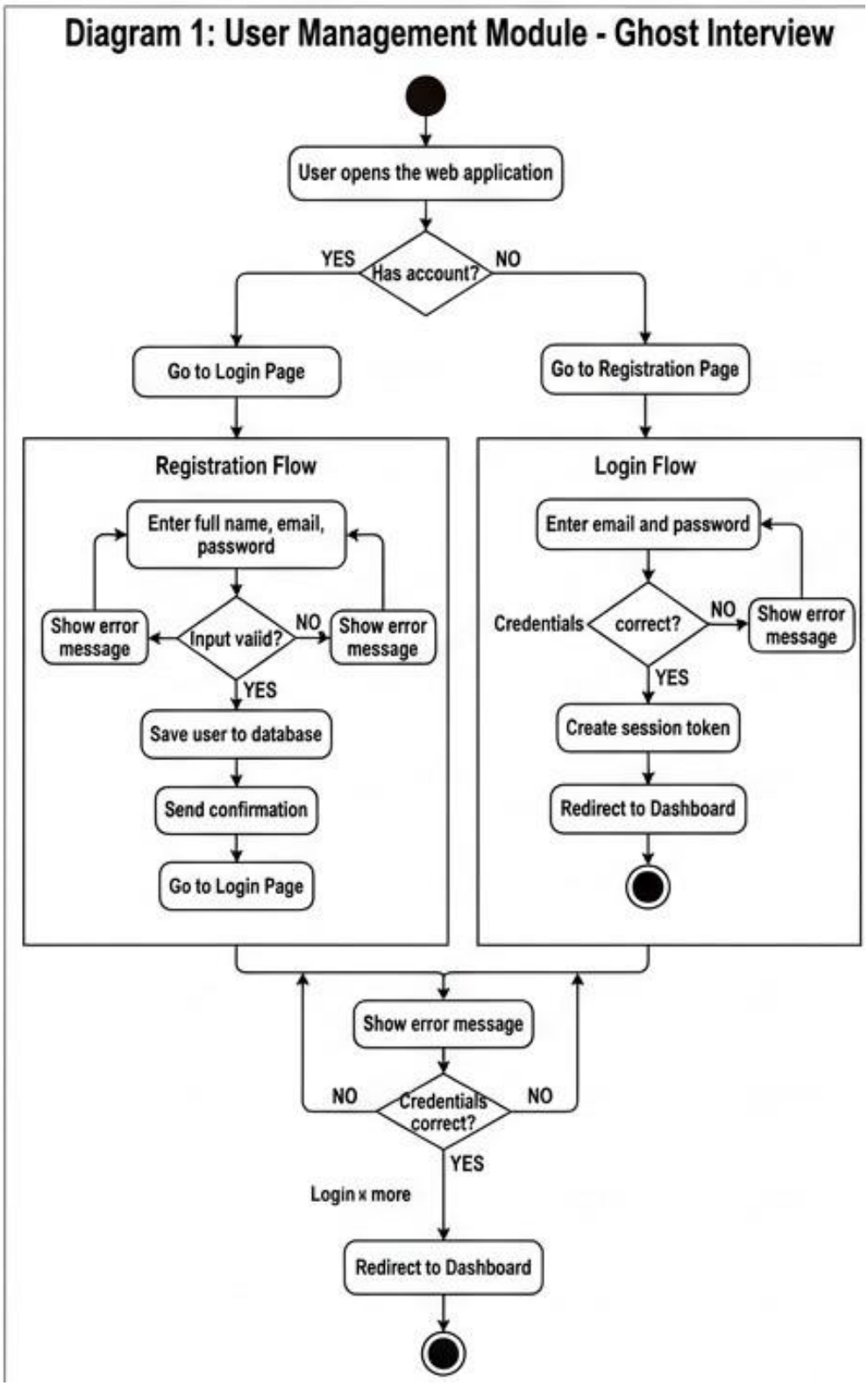


Figure 4.3: Activity Diagram User Management Module

5. User speaks their answer → Audio captured → Sent to Whisper for transcription.
6. Webcam frames captured simultaneously → Sent to DeepFace/Hume AI for emotion analysis.

7. System checks if all questions are answered → If yes, session ends and report is generated → If no, next question is delivered.

Module 3: Admin Module

Description: This diagram illustrates the administrator's operations within the Ghost Interview system.

Flow:

1. Admin logs in with elevated credentials.
2. Admin accesses the admin panel.
3. Admin can monitor registered users and their interview history.
4. Admin can manage job roles and interview question categories.
5. Admin can generate system-wide reports and analytics.
6. Admin can deactivate or remove user accounts.
7. Admin logs out.

4.5.2 Class Diagram

The class diagram shows the static structure of the Ghost Interview system, including all major classes, their attributes, methods, and relationships.

Key Classes:

User: Stores user pro le and authentication data. Methods include register(), login(), logout(), updateProfile().

InterviewSession: Manages a single interview session. Attributes include session ID, job role, start/end time, and status. Methods include startSession(), endSession(), getQuestions().

SpeechAnalyzer: Handles audio processing. Methods include captureAudio(), transcribeSpeech(), detectFillerWords().

EmotionAnalyzer: Handles webcam frame analysis. Methods include captureFrame(), analyzeEmotion(), getSentiment().

Report: Generates and stores performance reports. Methods include generateReport(), calculateConfidenceScore(), displayReport().

Admin: Extends User with elevated privileges. Methods include manageUsers(), manageJobRoles(), viewSystemAnalytics().

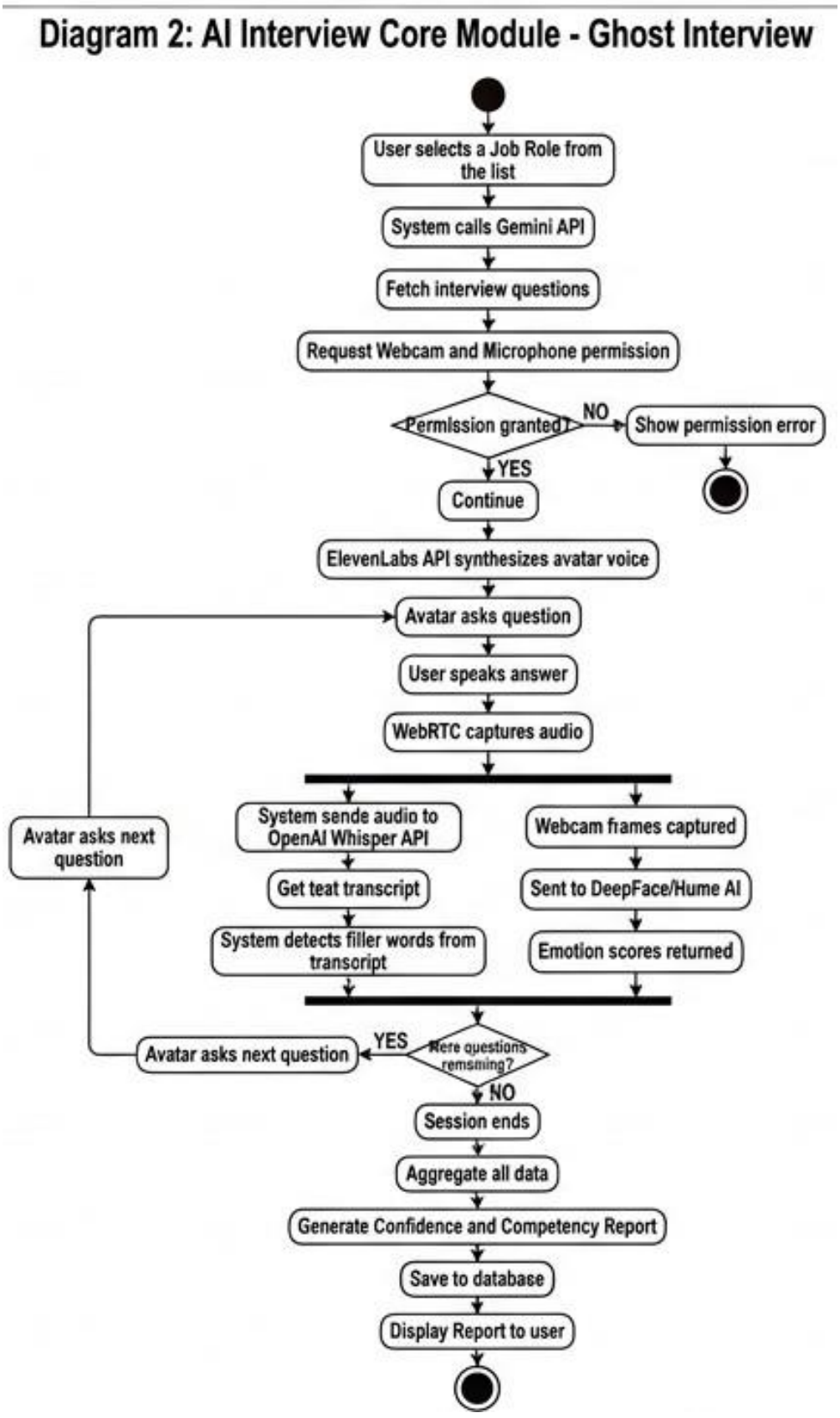


Figure 4.4: Activity Diagram AI Interview Core Module

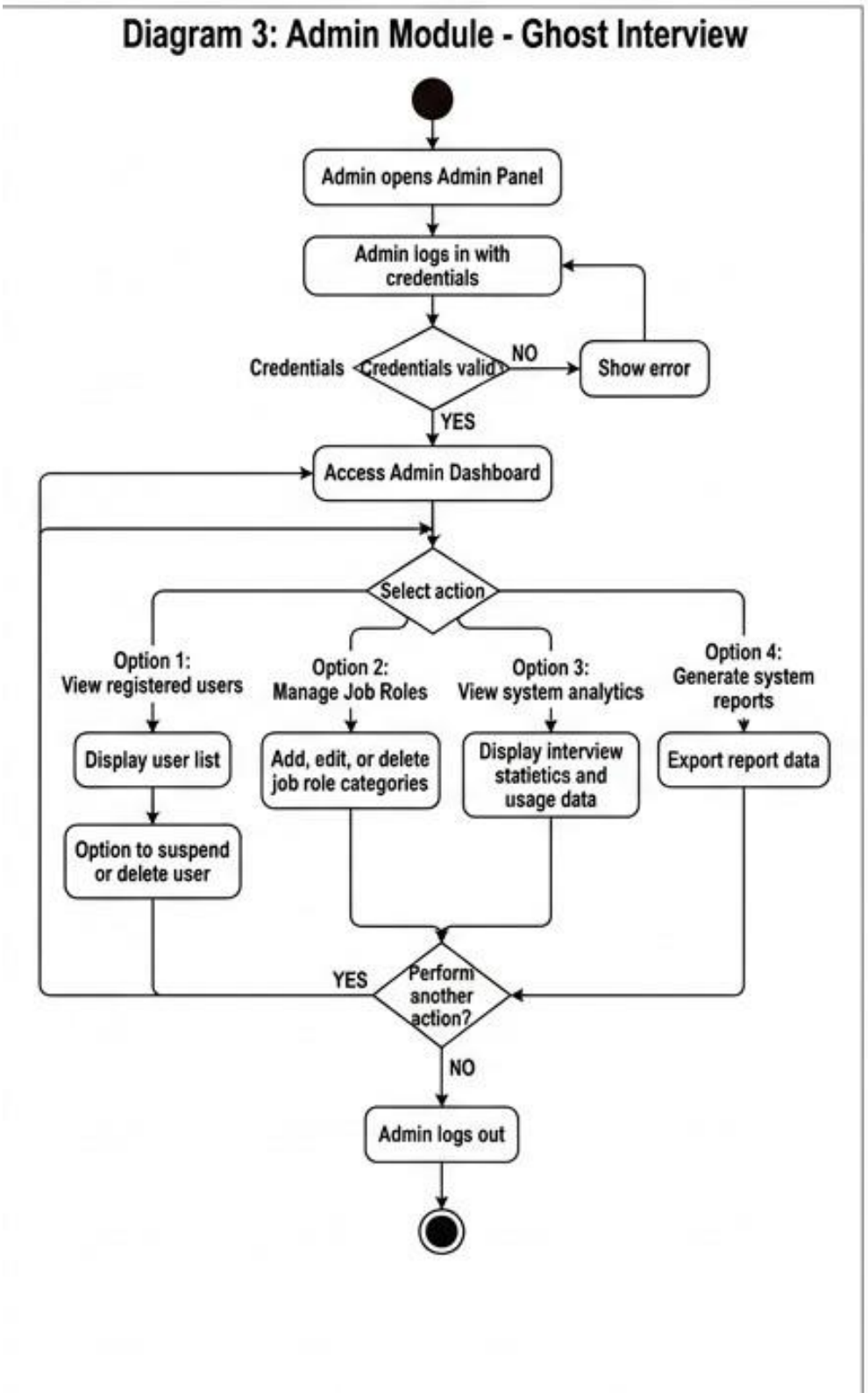


Figure 4.5: Activity Diagram Admin Module

JobRole: Stores prede ned job role categories and associated question sets.

Relationships:

User has many InterviewSessions (1:N).

InterviewSession uses SpeechAnalyzer and EmotionAnalyzer (Association).

InterviewSession generates one Report (1:1).

Admin inherits from User (Inheritance).

InterviewSession is associated with one JobRole (Association).

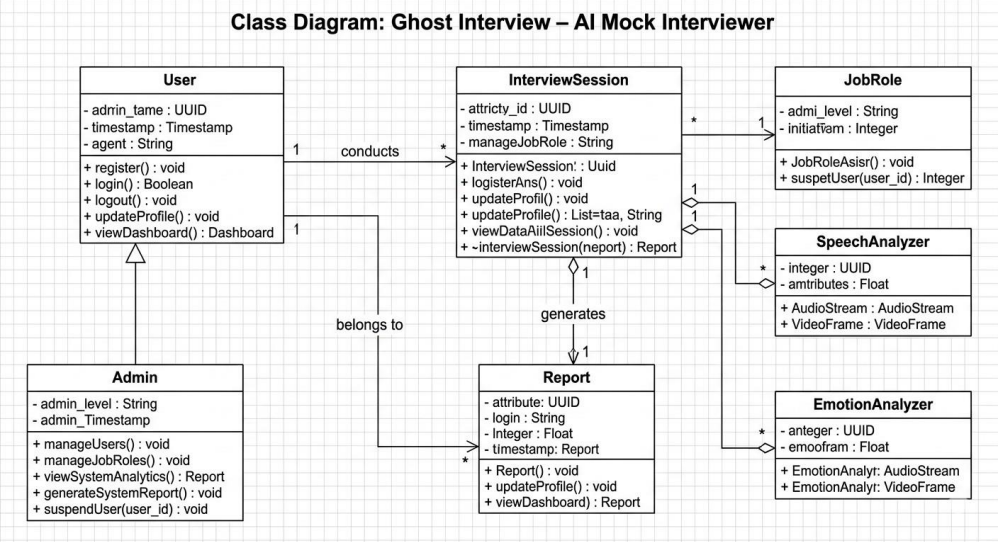


Figure 4.6: Class Diagram of Ghost Interview System

4.5.3 Sequence Diagrams

Sequence diagrams show the interaction between objects over time for major system operations.

Sequence Diagram User Login

Description: This sequence diagram shows the login ow for a registered user.

1. User enters email and password on the login page.
2. Frontend sends a POST request to the Backend (/api/login).
3. Backend queries the Database for user credentials.
4. Database returns user record.
5. Backend validates password hash.
6. If valid: Backend creates a session token and returns success response to Frontend.
7. Frontend redirects user to the dashboard.

8. If invalid: Backend returns an error response and Frontend displays error message.

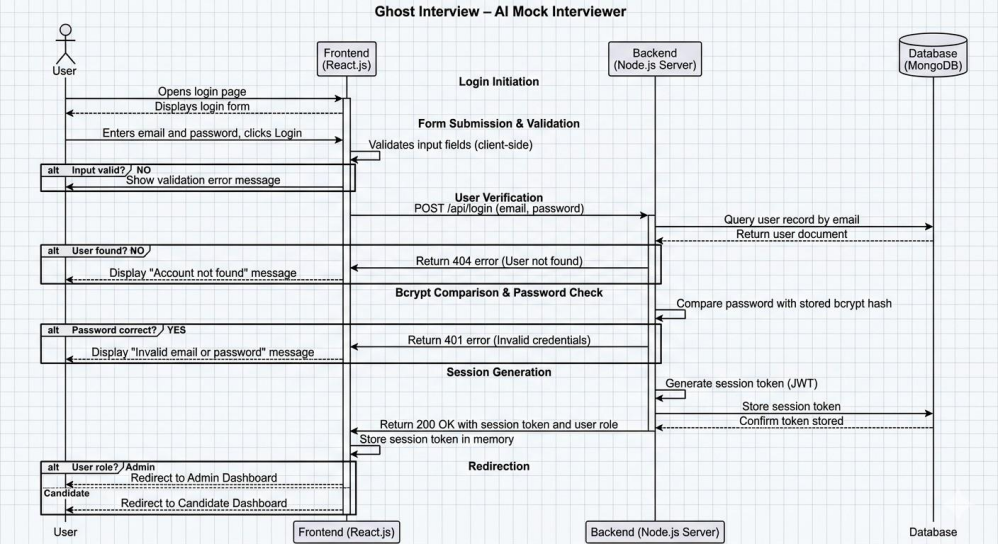


Figure 4.7: Sequence Diagram User Login

Sequence Diagram AI Interview Core Operation

Description: This sequence diagram shows the complete interview session ow from job role selection to report generation.

1. User selects a job role → Frontend sends request to Backend.
2. Backend calls Gemini API → Gemini returns interview questions.
3. Backend sends rst question text to Frontend.
4. Frontend calls ElevenLabs API → ElevenLabs returns audio stream → AI avatar speaks the question.
5. User speaks answer → WebRTC captures audio.
6. Frontend sends audio to Backend → Backend calls Whisper API → Whisper returns text transcript.
7. Backend detects ller words from transcript and stores result.
8. Simultaneously, Frontend captures webcam frames → Backend calls DeepFace/Hume AI → Emotion scores returned and stored.
9. Process repeats for all questions.
10. On session end: Backend aggregates all data → Generates Con dence & Competency Report → Stores in Database → Returns report to Frontend.
11. Frontend displays the report to the user.

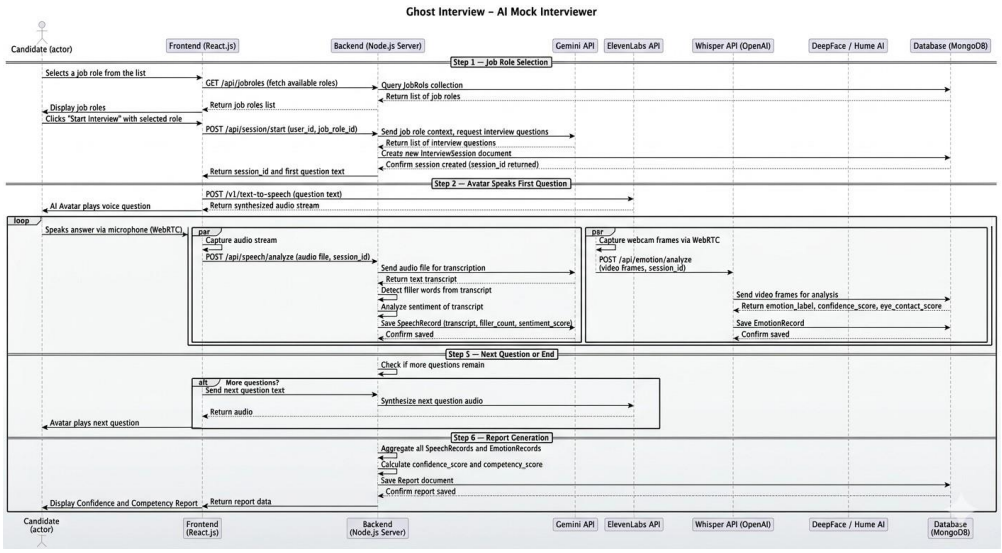


Figure 4.8: Sequence Diagram AI Interview Core Operation

4.5.4 State Transition Diagrams

State diagrams show how the system or an object changes states over time based on events and conditions.

Interview Session States:

Idle → (User selects job role) → Initializing

Initializing → (APIs loaded, webcam granted) → Active

Active → (All questions answered) → Processing

Processing → (Report generated) → Completed

Active → (User quits early) → Terminated

Active → (API failure) → Error

Error → (User retries) → Initializing

User Account States:

Registered → Active → Suspended → Deleted

4.6 Data Flow Diagrams

Data Flow Diagrams (DFD) describe how data moves within the Ghost Interview system. Di erent levels of DFD provide increasing detail.

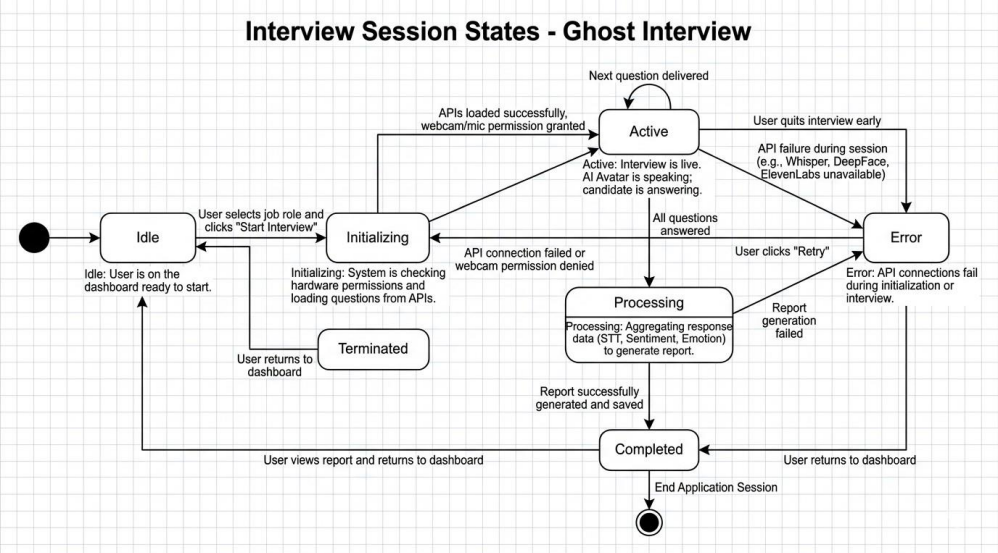


Figure 4.9: State Transition Diagram Interview Session

4.6.1 Level 1 Data Flow Diagram

Level 1 DFD decomposes the Ghost Interview system into its major sub-processes and shows how data ows between them.

Major Processes:

P1 Authentication Module: Handles user registration and login. Data ows: User credentials → P1 → User Database.

P2 Interview Engine Module: Manages job role selection and avatar interaction. Data ows: Job role → P2 → Gemini API → Questions → ElevenLabs → Audio.

P3 Speech Analysis Module: Processes audio responses. Data ows: Audio → P3 → Whisper API → Transcript → Session Store.

P4 Emotion Detection Module: Processes webcam frames. Data ows: Frames → P4 → DeepFace/Hume AI → Emotion scores → Session Store.

P5 Reporting Module: Generates performance reports. Data ows: Session Store → P5 → Report Database → User Dashboard.

P6 Admin Module: System monitoring and management. Data ows: Admin → P6 → User Database / Report Database.

4.6.2 Level 2 Data Flow Diagram

Level 2 DFD further decomposes the AI Interview Engine Module (P2) from Level 1 into detailed sub-processes.

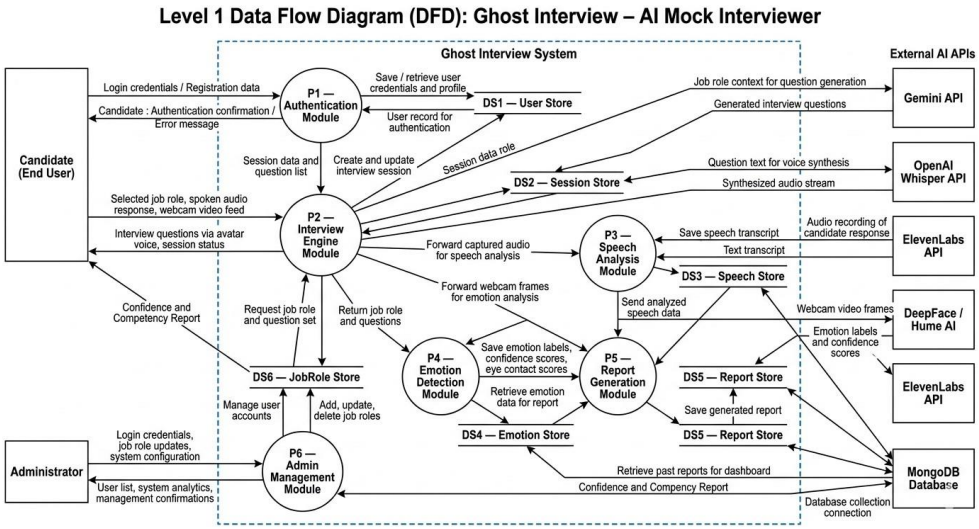


Figure 4.10: Level 1 Data Flow Diagram Ghost Interview System

Sub-processes of P2 Interview Engine:

P2.1 Job Role Loader: Fetches prede ned job roles from the database and presents them to the user.

P2.2 Question Generator: Sends job role data to Gemini API and retrieves a list of interview questions.

P2.3 Avatar Voice Synthesizer: Sends question text to ElevenLabs API and plays back the generated audio through the frontend.

P2.4 Response Collector: Captures user's spoken response via WebRTC microphone and forwards audio to the Speech Analysis Module (P3).

P2.5 Session Controller: Tracks question index, manages timing, and triggers session end when all questions are completed.

4.7 Data Design

This section describes how data is structured, stored, and managed within the Ghost Interview system.

Database Type: The system uses a NoSQL database (MongoDB) for the following reasons:

Interview session data and emotion metrics have variable and nested structures that are better represented as JSON documents.

MongoDB's exible schema accommodates future additions to report structure without migrations.

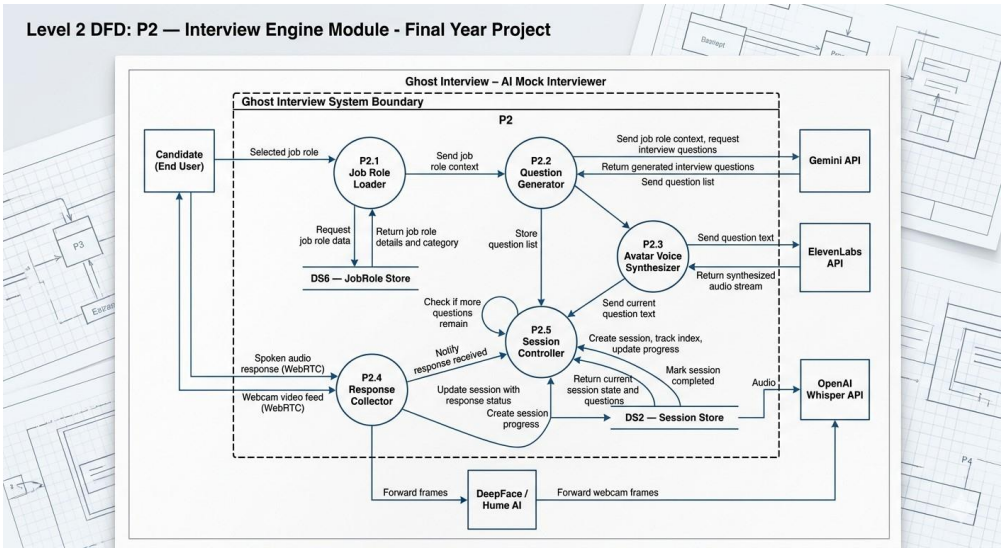


Figure 4.11: Level 2 Data Flow Diagram AI Interview Engine Module

High read/write throughput supports real-time session data storage during active interviews.

Indexing Strategy: Indexes are applied on user_id, session_id, and created_at elds for e cient querying of user sessions and reports.

Security: Passwords are hashed using bcrypt. All API keys are stored as environment variables server-side. Sensitive session data is accessible only to the authenticated user.

Scalability: MongoDB supports horizontal scaling through sharding. Collections are designed to support large volumes of transcript and emotion metric documents.

4.7.1 Entity Relationship Diagram (ERD)

The Entity Relationship Diagram represents the logical data structure of Ghost Interview.

Entities:

User (user_id PK, full_name, email, password_hash, role, created_at, status)

InterviewSession (session_id PK, user_id FK, job_role_id FK, start_time, end_time, status)

JobRole (job_role_id PK, role_name, category, description)

SpeechRecord (speech_id PK, session_id FK, transcript, ller_word_count, sentiment_score, recorded_at)

EmotionRecord (emotion_id PK, session_id FK, emotion_label, con dence_score, eye_contact_score, captured_at)

Report (report_id PK, session_id FK, user_id FK, con dence_score, competency_score, ller_count, emotion_summary, generated_at)

Relationships:

User conducts many InterviewSessions (1:N).

InterviewSession belongs to one JobRole (N:1).

InterviewSession has one SpeechRecord per question (1:N).

InterviewSession has many EmotionRecords (1:N).

InterviewSession generates one Report (1:1).

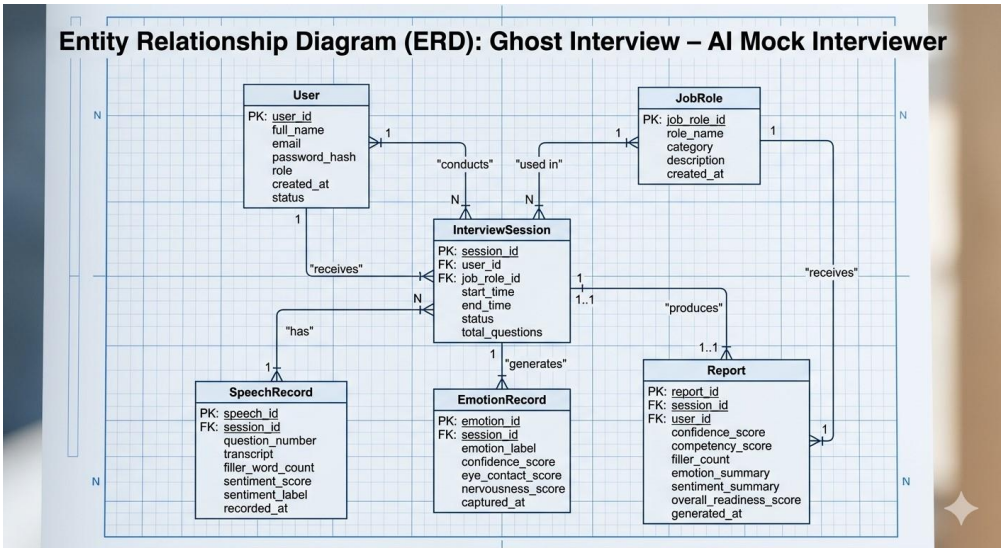


Figure 4.12: Entity Relationship Diagram Ghost Interview System

4.7.2 Database Architecture

The Ghost Interview system uses a Client-Server Architecture with the following database design:

Client-Server Architecture: The React.js frontend communicates with the Node.js backend server, which in turn queries the MongoDB database.

Cloud Storage: The database is hosted on MongoDB Atlas (cloud-managed), ensuring high availability and automated backups.

Session Data: Interview session documents, transcripts, and emotion records are stored as nested JSON documents within MongoDB collections.

4.8 Database Schema Diagram

The Database Schema Diagram represents the physical database implementation structure of the Ghost Interview system.

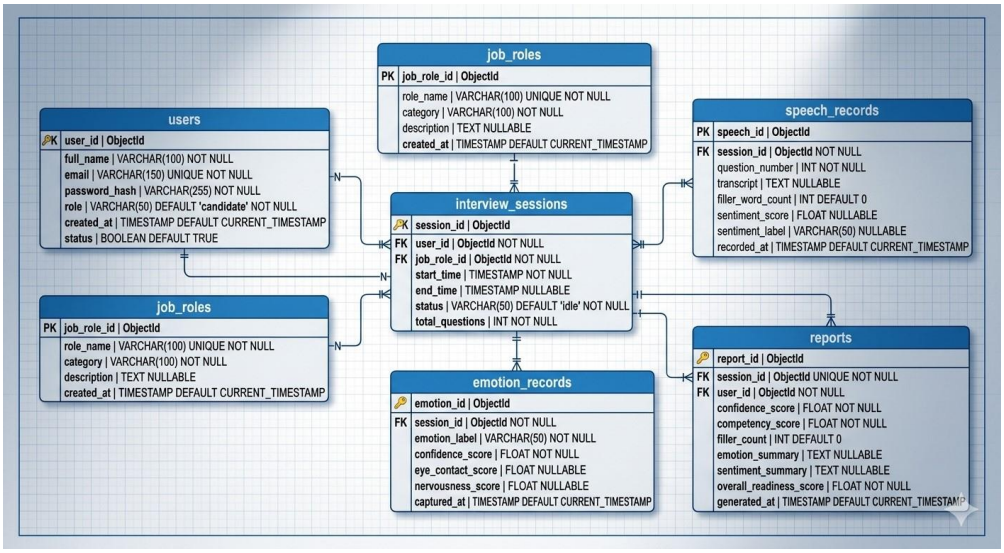


Figure 4.13: Database Schema Diagram Ghost Interview System

The schema is designed to support:

Data Integrity: Foreign key-equivalent references between collections ensure consistent relationships between users, sessions, and reports.

E cient Querying: Indexed elds on user_id and session_id reduce query response times.

Scalability: Document-based storage allows horizontal scaling via MongoDB Atlas sharding.

Security: Password elds store only bcrypt hashes; API keys are never stored in the database.

4.8.1 Data Dictionary

The Data Dictionary de nes the structure of all major data elements in the system.

4.9 Database Tables / Collections

This section describes the logical database design of the Ghost Interview system. The database is structured to ensure data consistency, integrity, security, and scalability.

Table 4.1: Data Dictionary User Collection

Field Name	Type	Size	Description

user_id	UUID / Ob-jectId		Unique identi er for each user (Primary Key)
full_name	VARCHAR	100	Full name of the registered user
email	VARCHAR	150	Unique registered email address used for login
password_hash	VARCHAR	255	Bcrypt-hashed password for secure authentication
role	VARCHAR	50	Access level: Admin or Candidate
created_at	TIMESTAMP		Account creation date and time
status	BOOLEAN		Account active (true) or suspended (false)

Table 4.2: Data Dictionary InterviewSession Collection

Field Name	Type	Size	Description
session_id	UUID / Ob-jectId		Unique session identi er (Primary Key)
user_id	UUID / Ob-jectId		Reference to the User collection (Foreign Key)
job_role_id	UUID / Ob-jectId		Reference to the JobRole collection (Foreign Key)
start_time	TIMESTAMP		Timestamp when the interview session started
end_time	TIMESTAMP		Timestamp when the interview session ended
status	VARCHAR	50	Session state: Active, Completed, Terminated, Error

Table 4.3: Data Dictionary SpeechRecord Collection

Field Name	Type	Size	Description
speech_id	UUID / Ob-jectId		Unique speech record identi er (Primary Key)
session_id	UUID / Ob-jectId		Reference to InterviewSession (Foreign Key)
transcript	TEXT		Full text transcript of the user's spoken answer
ller_word_count	INT		Total number of ller words detected in the response
sentiment_score	FLOAT		Sentiment polarity score (positive/neutral/negative)
recorded_at	TIMESTAMP		Timestamp of the speech recording

Table 4.4: Data Dictionary EmotionRecord Collection

Field Name	Type	Size	Description
emotion_id	UUID / Ob-jectId		Unique emotion record identi er (Primary Key)
session_id	UUID / Ob-jectId		Reference to InterviewSession (Foreign Key)
emotion_label	VARCHAR	50	Detected emotion: Happy, Neutral, Nervous, Angry, etc.
con dence_score	FLOAT		AI model con dence score for the detected emotion
eye_contact_score	FLOAT		Score indicating level of eye contact with camera
captured_at	TIMESTAMP		Timestamp when the webcam frame was captured

Table 4.5: Data Dictionary Report Collection

Field Name	Type	Size	Description
report_id	UUID / Ob-jectId		Unique report identi er (Primary Key)
session_id	UUID / Ob-jectId		Reference to InterviewSession (Foreign Key)
user_id	UUID / Ob-jectId		Reference to the User collection (Foreign Key)
con dence_score	FLOAT		Overall con dence score computed from emotion data
competency_score	FLOAT		Overall competency score based on speech quality
ller_count	INT		Total ller words across the entire session
emotion_summary	TEXT		JSON summary of emotion distribution during session
generated_at	TIMESTAMP		Timestamp when the report was generated

4.9.1 User Collection

The User collection stores authentication credentials and user pro le information. It supports role-based access control and secure authentication. Primary Key: user_id

Relationships: Linked to InterviewSession and Report collections.

Authentication Attributes: Email, password hash, role, account status.

Table 4.6: User Collection Structure

Field Name	Data Type	Constraint	Description
user_id	UUID / Objec-tId	Primary Key	Unique identi er for each user
full_name	VARCHAR(100)	Not Null	User's full name
email	VARCHAR(150)	Unique, Not Null	Login email address
password_hash	VARCHAR(255)	Not Null	Bcrypt-encrypted password
role	VARCHAR(50)	Not Null	Admin or Candidate
created_at	TIMESTAMP	Default Current	Account creation timestamp
status	BOOLEAN	Default True	Active or suspended account

4.9.2 Core Data Collections

These collections store the primary operational data processed by the Ghost Interview system.

InterviewSession Collection Stores metadata for each mock interview session conducted by a user.

JobRole Collection Stores prede ned job categories (e.g., Software Engineer, Data Analyst, HR Manager) and associated interview question sets.

SpeechRecord Collection Stores the text transcript, ller word count, and sentiment score for each question's spoken response.

EmotionRecord Collection Stores facial emotion analysis results captured from webcam frames at regular intervals during the session.

Report Collection Stores the nal Con dence and Competency Report generated after each completed interview session.

4.9.3 Relationships Between Collections

One-to-Many (1:N): One User may conduct many InterviewSessions.

One-to-Many (1:N): One InterviewSession has many SpeechRecords (one per question) and many EmotionRecords (one per frame batch).

One-to-One (1:1): One InterviewSession generates exactly one Report.

Many-to-One (N:1): Many InterviewSessions may use the same JobRole.

4.9.4 Security and Data Integrity

To ensure con dentiality and integrity of stored data, the following measures are implemented:

Password Encryption: Passwords are hashed using bcrypt before storage.

Role-Based Access Control (RBAC): Admins and Candidates have distinct access privileges enforced at the API layer.

Input Validation: All user inputs are validated both client-side and server-side before database operations.

HTTPS Communication: All data transmission between client and server uses HTTPS encryption.

API Key Security: Third-party API keys (Gemini, Whisper, ElevenLabs, DeepFace) are stored as server-side environment variables and never exposed to the client.

Database Backup: MongoDB Atlas provides automated daily backups and point-in-time recovery.

4.9.5 Scalability Considerations

The database design incorporates scalability mechanisms to support future growth:

Horizontal Scaling: MongoDB Atlas supports sharding for distributing large datasets across multiple nodes.

Indexing: Indexes on user_id, session_id, and created_at improve query performance for dashboard and report retrieval.

Caching: Frequently accessed data such as job role lists can be cached using Redis to reduce database load.

Load Balancing: The Node.js backend can be deployed across multiple instances behind a load balancer for high availability.

4.10 Chapter Summary

This chapter presented the comprehensive software design of the Ghost Interview AI Mock Interviewer system. It translated the functional and non-functional requirements identi ed in Chapter 3 into structured architectural, behavioral, and data models.

The Object-Oriented Design methodology combined with a Layered Architecture was justi ed as the most suitable approach for this AI-integrated web application. The Agile

Incremental process model was selected to accommodate iterative re nement of AI service integrations.

A high-level system overview was provided through the context diagram and threetier architectural representation, clearly illustrating interactions between the React.js frontend, Node.js backend, four AI service APIs, and the MongoDB database.

Detailed UML models including activity diagrams for all three modules, class diagram, sequence diagrams for login and interview operations, and state transition diagrams were developed to describe system behavior and structure. Data ow diagrams at Level 1 and Level 2 illustrated how data moves across the system during interview sessions.

The chapter further elaborated the database design through an Entity Relationship Diagram, database schema diagram, and comprehensive data dictionaries for all ve major collections. Security measures, scalability strategies, and data integrity considerations were also addressed.

Overall, this chapter establishes a clear and detailed technical blueprint that guides the implementation phase of the Ghost Interview system.